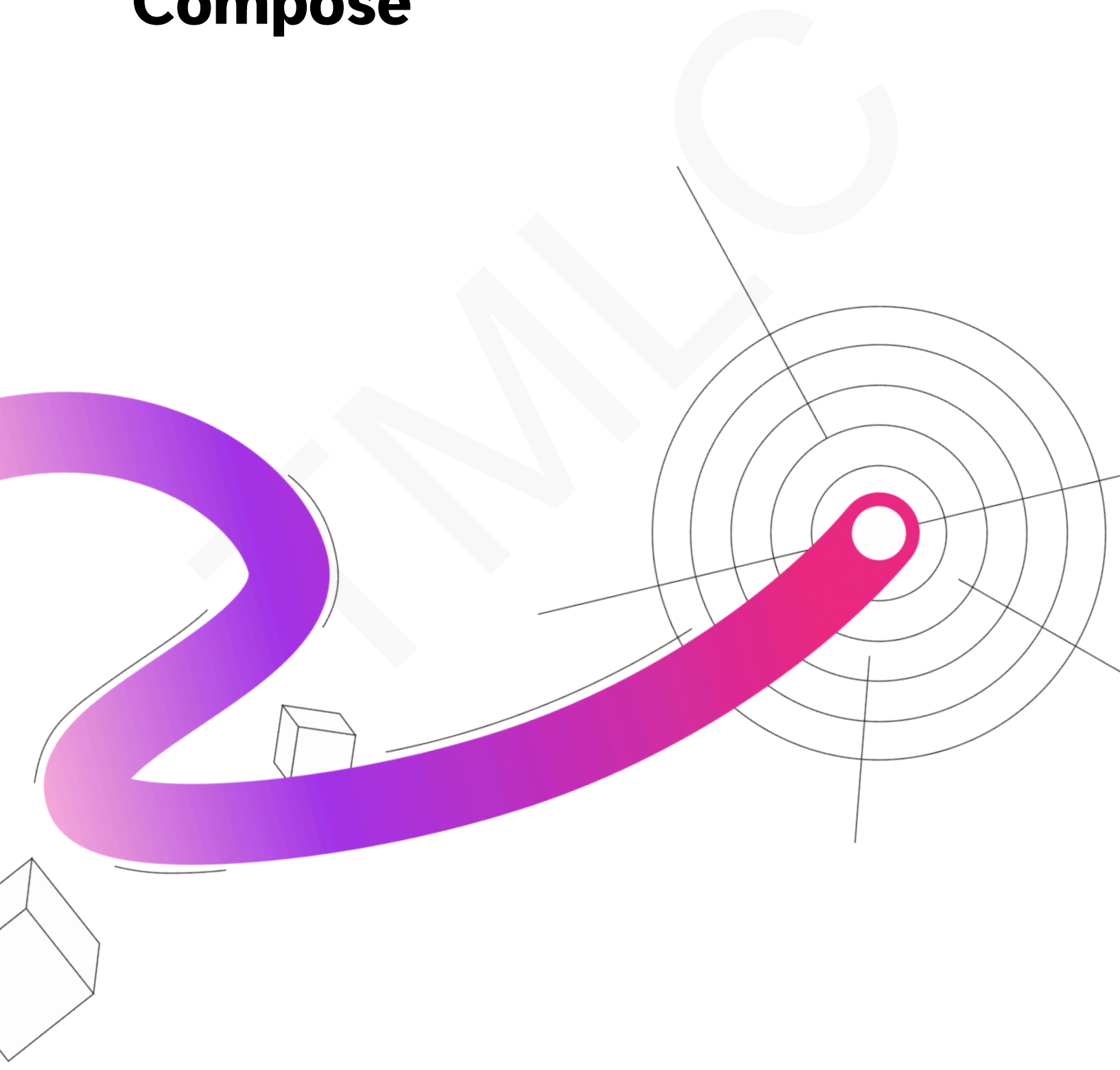


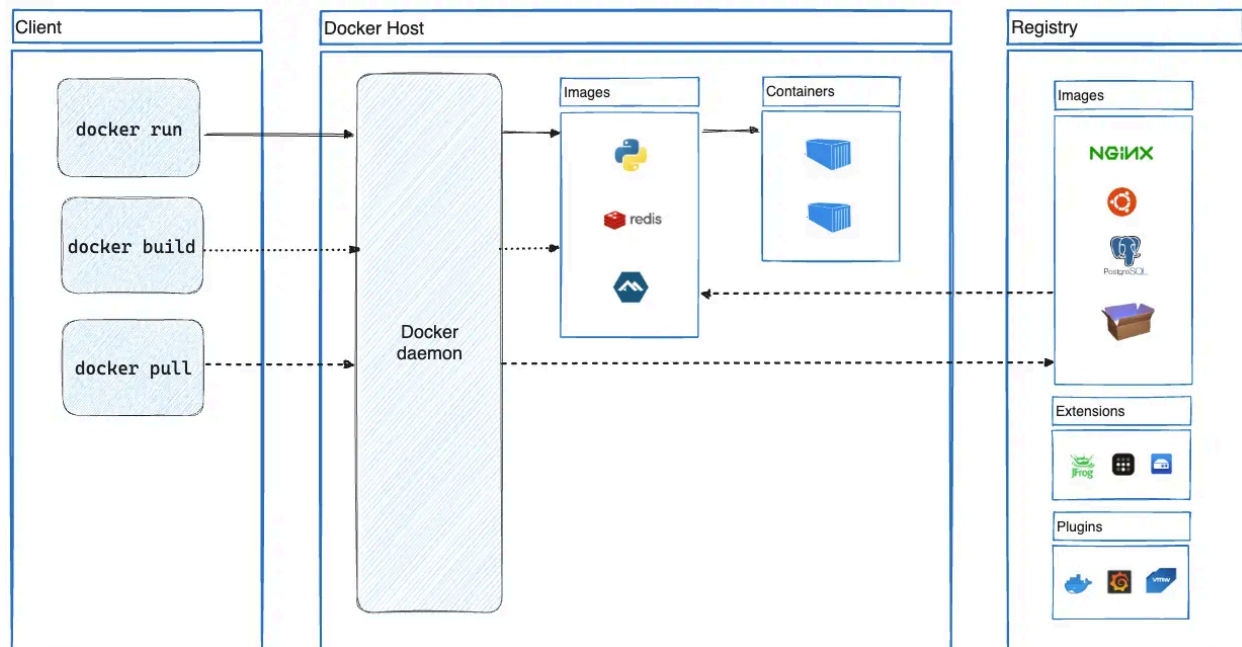
Docker and Docker Compose



In the world of Generative AI (GenAI), where models range from closed-source giants to open-source alternatives, setting up consistent and reproducible environments is crucial. Enter **Docker** and **Docker Compose**—two powerful tools that streamline development, deployment, and collaboration. Whether you're working with OpenAI's GPT models, Meta's Llama, or self-hosted AI solutions, Docker simplifies infrastructure management.

What is Docker?

Docker is a **containerization platform** that allows developers to package applications and their dependencies into isolated units called **containers**. Unlike traditional virtual machines (VMs), Docker containers share the host operating system's kernel, making them lightweight and fast.



Read more about Daemon, Client and Desktop on this [link](#).

Why Use Docker in AI Development?

- **Consistency Across Environments:** No more "it works on my machine!" Containers ensure uniform behavior across different systems.
- **Dependency Management:** AI projects rely on specific Python versions, CUDA configurations, and library dependencies. Docker encapsulates these for seamless execution.
- **Portability & Scalability:** Easily deploy AI models on local machines, cloud servers, or Kubernetes clusters.
- **Resource Efficiency:** Unlike VMs, containers require fewer system resources, allowing you to run multiple applications without significant performance loss.
- **Security & Isolation:** Docker ensures that applications and their dependencies are isolated from the host system, reducing security vulnerabilities.

Understanding Images and Containers

Docker Images

A **Docker image** is a lightweight, standalone, and executable software package that includes everything needed to run a piece of software, including the code, runtime, libraries, and dependencies. Images act as blueprints for containers.

Common Docker Image Commands

- **List available images:**

```
docker images
```

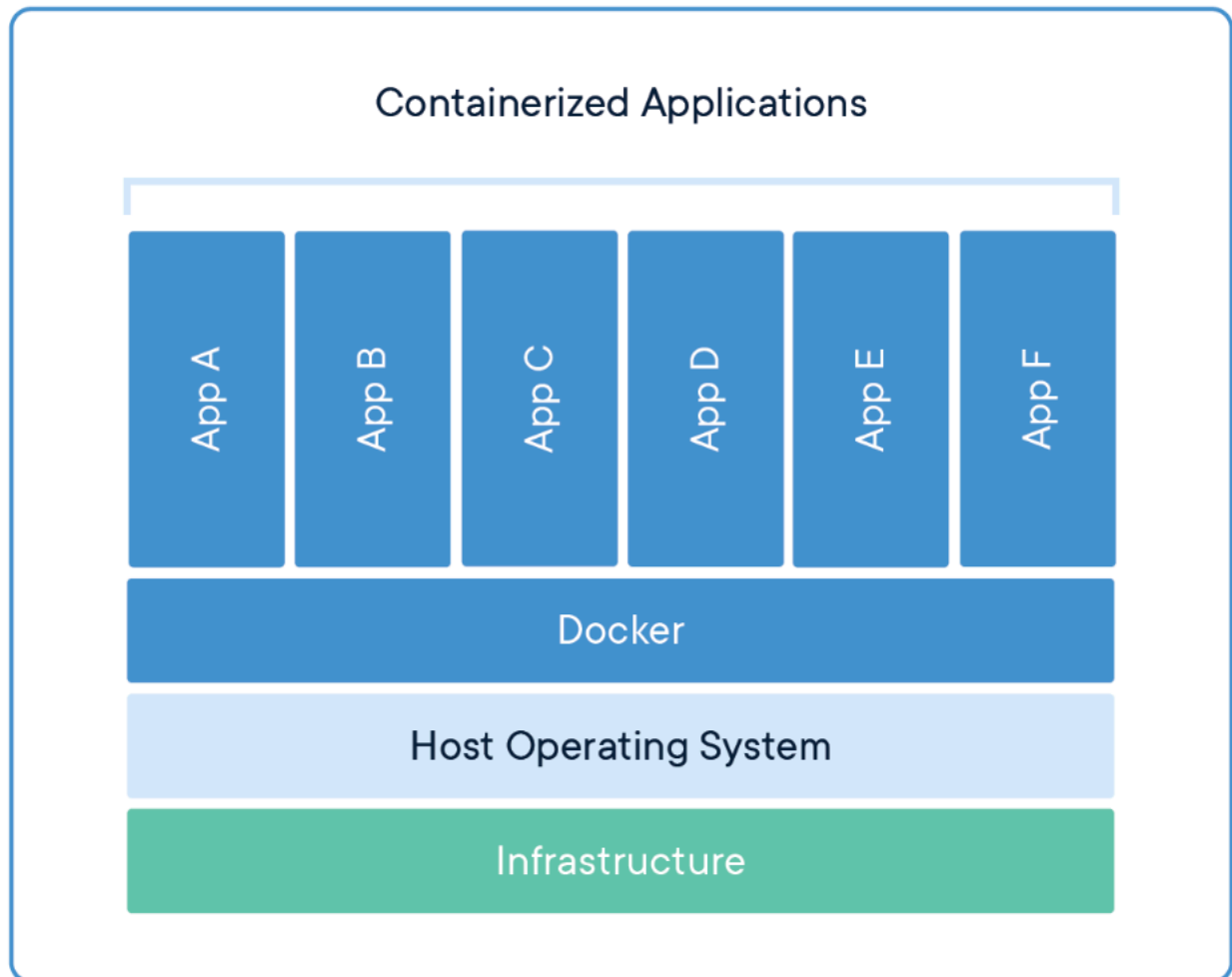
- **Pull an image from Docker Hub:**

```
docker pull ubuntu:latest
```

- **Remove an image:**

```
docker rmi ubuntu:latest
```

Docker Containers



A **Docker container** is an instance of a Docker image that runs as an isolated process on the host system. Containers use the underlying image but are ephemeral, meaning they can be started, stopped, or removed easily.

Common Docker Container Commands

- **Run a container interactively:**

```
docker run -it ubuntu:latest /bin/bash
```

- **List running containers:**

```
docker ps
```

- **List all containers (including stopped ones):**

```
docker ps -a
```

- **Stop a running container:**

```
docker stop <container_id>
```

- **Remove a stopped container:**

```
docker rm <container_id>
```

Getting Started with Docker

Installing Docker: Docker can be installed from [docker.com](https://docs.docker.com/engine/install/). Once installed, verify the installation using:

```
docker --version
```

Standard Docker Commands

Here are some fundamental Docker commands to get you started:

- **Pull an image:**

```
docker pull python:3.9
```

- **Run a container interactively:**

```
docker run -it --rm python:3.9
```

- **List running containers:**

```
docker ps -a
```

- **Remove an image:**

```
docker rmi python:3.9
```

Please visit this [link](#) to check all Docker commands

Creating a Docker Image for a FastAPI Application

Let's build a simple **FastAPI** application and package it into a Docker image.

Step 1: Create a FastAPI Application

Create a new directory and a Python file named **main.py**:

```
from fastapi import FastAPI

app = FastAPI()
```

```
@app.get("/")
def read_root():
    return {"message": "Hello, Docker with FastAPI!"}
```

Step 2: Create a **requirements.txt** File

Specify dependencies in a **requirements.txt** file:

```
fastapi
uvicorn
```

Step 3: Create a **Dockerfile**

Define how the FastAPI application should be containerized:

```
# Use the official Python image as a base
FROM python:3.9

# Set the working directory
WORKDIR /app

# Copy the application files
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose the application port
EXPOSE 8000

# Command to run the FastAPI application
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 4: Build and Run the Docker Image

Build the Docker Image

```
docker build -t fastapi-app .
```

Run the Container

```
docker run -d -p 8000:8000 fastapi-app
```

Now, your FastAPI application is running inside a Docker container, accessible at <http://localhost:8000>.

What is Docker Compose?

While Docker is great for managing individual containers, **Docker Compose** allows you to define and run multi-container applications using a simple **YAML file**. This is especially useful in AI projects where you might need a model server, database, and a front-end interface.

Standard Docker Compose Commands

Run where .yaml is located

- **Start services in the background:**

```
docker-compose up -d
```

- **Stop all services:**

```
docker-compose down
```

- **View running services:**

```
docker-compose ps
```

- **View logs of services:**

```
docker-compose logs
```

- **Restart services:**

```
docker-compose restart
```

Example: AI Model Deployment with Docker Compose

Let's say you want to deploy an **LLM API** with a **vector database** (like ChromaDB) and a **FastAPI backend**. Here's a `docker-compose.yaml` file to achieve that:

```
version: '3.8'
services:
  model:
    image: ollama/ollama
    ports:
      - "8001:8001"
    volumes:
      - ./models:/models

  chromadb:
    image: chromadb/chromadb
    ports:
      - "8002:8000"

  backend:
```

```
build: ./backend
ports:
  - "8000:8000"
depends_on:
  - model
  - chromadb
```

Running the Stack

After saving the above YAML file, start everything with:

```
docker-compose up -d
```

This launches all services in detached mode. Now your AI model, database, and backend are running together seamlessly.

Conclusion

Docker and Docker Compose empower GenAI developers by simplifying complex setups, ensuring reproducibility, and enabling smooth collaboration. Whether you're running closed-source models or experimenting with open-source alternatives, **containerization accelerates AI development**. Start using Docker today!